Cams4Bikes: Reliable and Privacy-Preserving Video Handling for Shared Bikes

Jing Tan, Jenna Yuan

Why this subsystem exists

Shared bikes make transportation more accessible, but they also put riders in an exposed environment: busy roads, little physical protection, and limited accountability when something goes wrong. Newplace's camera pilot is meant to help on both sides. It gives riders a way to record their rides, and it gives Bikes4All a way to preserve accident footage for safety review and repair decisions. That opportunity also creates a new systems problem. Video is only useful if it gets to the right place reliably, and it is only acceptable if one rider's footage never leaks to an unintended user or outside the parts of the system that are authorized to handle it.

Our subsystem, Cams4Bikes, handles the camera pipeline for camera-equipped bikes: recording, transfer, temporary or durable storage, access control, and cleanup for both personal and accident videos. Two design properties drive the whole subsystem: reliability and privacy. Reliability matters because accident footage is a required system record. It must survive failures, retries, and rapid bike turnover. In our design, a bike returns to service only after required accident footage has been durably stored in the central computing facility (CCF) and all prior rider footage has been removed from the bike itself. That rule is stricter than the minimum requirement in the spec, but we chose it because it preserves rider isolation and makes reuse safer. Privacy matters because both personal and accident videos are tied to a specific rider and ride. Personal videos should exist only long enough to become available in the rider's app, while accident videos should remain protected even if the rider from that ride is later allowed to access them.

At a high level, Cams4Bikes uses a dock-first upload path, station buffering, chunked resumable transfer, strict return-to-service rules, and metadata-based rider isolation. Personal videos are delivered through the app and deleted from central storage after a fixed expiration window. Accident videos are durably stored in the CCF and can also be made visible in the app to the rider from that same ride. The groups most affected by this subsystem are current riders, future riders, and Bikes4All staff who rely on accident footage for operations and review. Current riders need confidence that their videos will actually appear in the app and that nobody else will see them. Future riders need accurate bike availability information, since video state can temporarily block reassignment. Operations staff need accident footage to survive failures. Those needs shaped the design choices we made.

System context and assumptions

Cams4Bikes sits inside the larger bikeshare system as the subsystem responsible for camera data and video-related state. It interacts with camera-equipped bikes, docks and the station computer, the CCF, the rider app, and the Reservations Team's modules only where video state affects the bike's eligibility. It does not redesign billing, the general checkout workflow, the kiosk UI, or the

reservation workflow itself. Instead, it provides video-related eligibility and restriction information to the rest of the system.

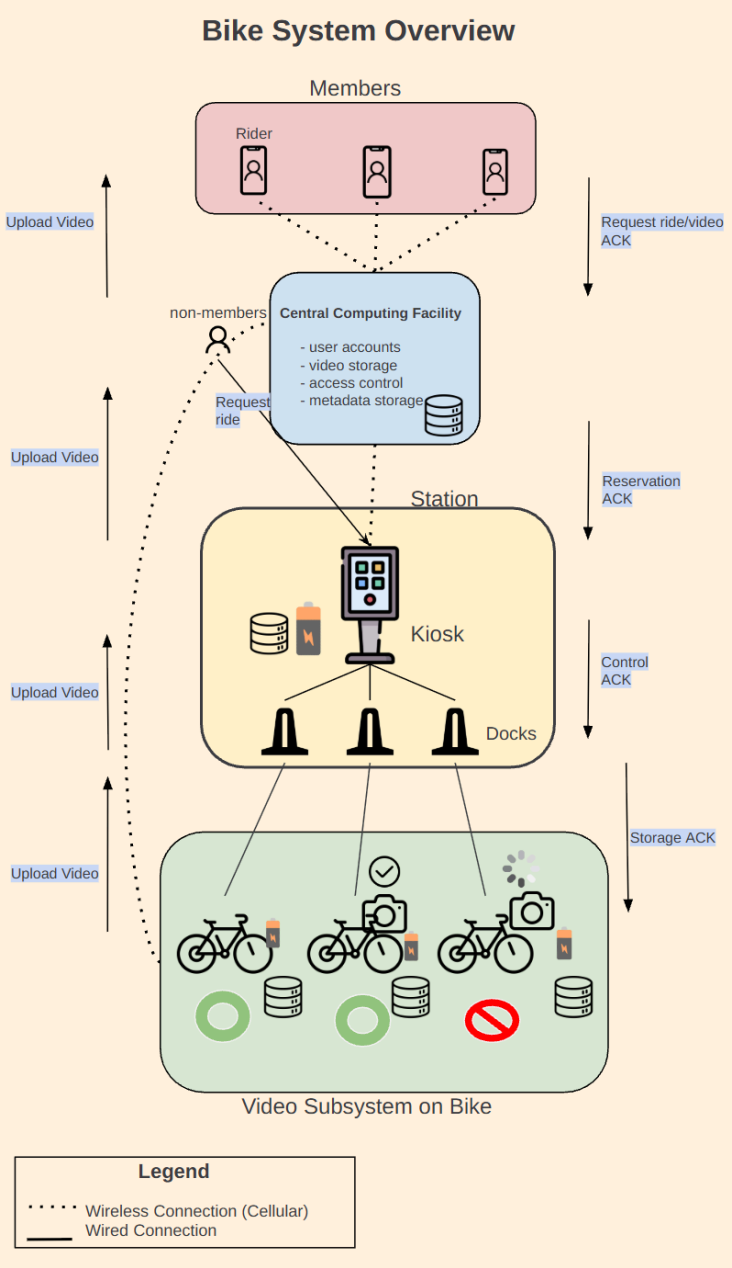

Figure 1. Cams4Bikes in the larger Bikes4All system. The figure shows how personal and accident footage move from the bike through the dock and station to the CCF, while status and acknowledgements flow back to the bikes.

A few system facts and design assumptions matter early because they shape almost every later decision. The following hardware and capacity facts come directly from the specification: cameras appear on about 10% of bikes system-wide, stations support 15 to 45 docks, each station has 1 TB of local storage, docked transfer uses USB 3.1 up to 600 MB/s, and station-to-central

transfer is limited by the station's 1 Gbps network interface. We treat the 10% camera-bike figure as a system-wide average, not a per-station cap, because bikes move and may temporarily collect at one station. For planning purposes, that means a 45-dock station would often have about 4 to 5 camera bikes, although the true number could temporarily be higher if camera-equipped bikes collect there. We also assume station storage is temporary buffer storage rather than archival storage, direct bike-to-CCF wireless upload is fallback only, accident videos take priority when resources are constrained, and personal videos are designed to appear in the rider's app through the CCF rather than requiring direct Bluetooth extraction from the bike.

A few terms also help make the design easier to follow. *Durable station receipt* means the station has safely buffered a chunk and can acknowledge it to the bike. *Safe persistence* for accident videos means the complete accident video has been durably stored in the CCF. A bike is *locally clean* when prior rider footage has been removed from on-bike storage. A bike is *ready for service* only when required accident footage is safely persisted and the bike is locally clean. An *accident event* is one accident-triggered recording that contains two synchronized streams, one forward-facing and one rider-facing. The *extraction window* is the limited post-docking period during which the bike finalizes and offloads the rider's last personal recording before local cleanup.

How the subsystem is organized

Within that setting, Cams4Bikes has five main modules. The *Bike Video Manager* lives on the bike. It stores personal and accident clips, tracks metadata such as bikeID, riderID, rideID, videoID, video type, timestamp, and accident-event information, and enforces deletion rules. It also tracks transfer progress and supports the post-docking extraction window for the rider's last personal recording. The *Dock Ingestion Manager* receives docked video over USB, accepts uploads in chunks identified by (videoID, chunkNumber), verifies each chunk with a checksum, and acknowledges durable station receipt. It also distinguishes accident from personal uploads so the rest of the system can prioritize them differently. The *Station Video Buffer* stores uploads temporarily and forwards them to the CCF. It maintains separate accident and personal queues, uses strict priority for accident uploads, and uses first-in, first-out ordering within each queue. It also preserves progress during station-network interruptions and tracks whether a video is only station-buffered or already safely persisted in the CCF. The *Central Video Service* receives station uploads and, when necessary, fallback wireless uploads. It deduplicates retries using the videoID and a record of which chunks have already been received, stores accident videos durably, and stores personal videos only temporarily so they can appear in the rider's app before expiring. It also serves videos only to authorized users. Finally, the Access Control and Availability Service does not replace the reservation subsystem but instead provides it with video-related state that may affect bike eligibility. It marks bikes unavailable when required accident uploads are incomplete or prior rider footage remains on-bike, exposes video and availability status to the bike screen, kiosk, and app, and ties those decisions to the correct rider and ride so that one rider cannot access another rider's videos.

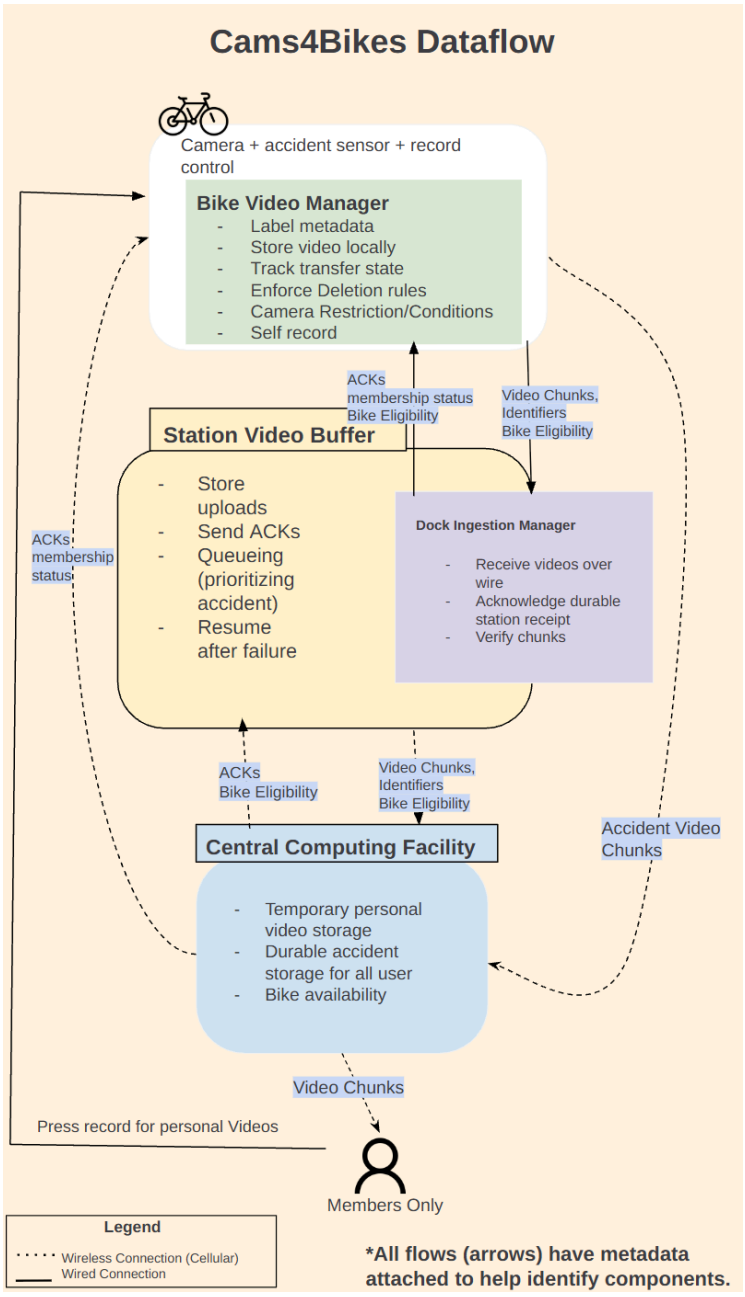

Figure 2. Internal architecture of the Cams4Bikes subsystem, showing the primary modules, their responsibilities, and the main data and control flows between the bike, station, CCF, and rider app.

The main design decisions

The most important design choice in the subsystem is our choice to use the docked station path as the default upload path for all routine transfers. This is the simplest and strongest path in the specification. USB transfer at 600 MB/s is much faster than the wireless options, which helps the station absorb moments when several camera bikes are returned around the same time. However,

forwarding from the station to the CCF is slower, which is why local station buffering is still necessary. Under a conservative assumption of one full 30-minute personal recording plus one 3-minute accident event, the bike may hold about 33 minutes of video, or about 89.1 GB at the given 45 MB/s accumulation rate. Uploading that full amount to the station takes about 148.5 seconds, or roughly 2.5 minutes.

The real bottleneck, though, is not bike-to-station transfer. It is accident-video forwarding from the station to the CCF, because that is what determines whether a bike can return to service. One accident event is about 8.1 GB, since it records for 3 minutes at the 45 MB/s total accumulation rate. If five camera bikes are returned close together and all five contain accident footage, the strict-priority accident queue drains to the CCF in about 5.4 minutes for the full set of five accident videos. If the accident clip is sent to the station first, the overall worst-case docking-to-eligibility time is about 5.6 minutes, with the extra 0.2 minutes coming from the initial bike-to-station transfer of that 8.1 GB clip. This is why station buffering matters: it lets the station absorb fast local uploads while slower central forwarding catches up, which directly determines how quickly bikes can return to service.

That same logic leads naturally to our second major choice: chunked, acknowledged, resumable transfer. Videos are split into chunks, each chunk includes identity information and a checksum, and the receiver acknowledges a chunk only after durable receipt. If a transfer is interrupted, the sender retransmits only missing chunks. This supports reliability in a few ways at once. It prevents silent corruption, avoids duplicate final objects, and lets progress resume rather than restart. In a system where bikes, stations, and central services may all fail independently, that ability to resume from a partially completed state matters a lot more than a simpler “all or nothing” model would.

The distinction between accident videos and personal videos is equally important. Accident videos are required system records. They must reach the CCF, they get higher forwarding priority, and they cannot be deleted before safe persistence. Personal videos are rider-facing videos rather than required operational records. Because personal videos are meant to be auto-delivered in the rider’s app, our design supports personal-video recording only for members. We move them to temporary central storage so they can appear in the rider’s app, and then delete them after expiration. This directly supports privacy because personal footage remains stored in the CCF only for as long as needed. Riders can view personal videos in the app and, in our design, may also view their own accident videos after those videos have been safely persisted in the CCF.

This distinction also shapes our return-to-service rule, which is intentionally conservative. A bike is not ready for reassignment if required accident uploads are incomplete or if prior rider footage still remains on-bike. Once accident footage is safely persisted in the CCF and the bike is locally clean, the bike can return to service. Personal videos may still be moving from station buffer to temporary central storage after that point, because that no longer threatens accident-video durability or rider isolation. That is one of the clearest places where our two design properties pull together: reliability comes from making sure required accident footage is safely persisted

before reuse, and privacy comes from ensuring prior rider footage is cleared from the bike before a new rider touches it.

Privacy is not just a retention policy, though. It also depends on metadata binding, access control, and encryption. Each video always stays bound to the correct rider and ride metadata. Personal videos are accessible only to that rider and remain in central storage only temporarily. Accident videos may be visible to the authenticated rider from that same ride only after safe persistence in the CCF. Communication between the station and the CCF and between the app and the CCF uses the latest version of Transport Layer Security 1.3, a standard method which protects data in transit from eavesdropping and tampering. Temporary stored video objects and sensitive metadata are also encrypted at rest using authenticated encryption.

Finally, another key part of the design is keeping user-visible status accurate. Riders and staff need to know whether a bike is available, unavailable, or camera-restricted, and whether all video has been transferred. Misleading status harms reliability, privacy, and rider trust because it leads to wrong assumptions about both bike readiness and video availability. Status has to show up on the bike screen, kiosk, and app for the rider to see.

Those choices imply a few guarantees we treat as invariants. Accident videos are never deleted before safe persistence in the CCF. One rider cannot access another rider's videos. A bike cannot become available while required accident uploads have not been safely persisted. In our design, a bike also cannot become available when it's not locally clean. Retrying an upload does not create duplicate stored videos, and user-visible status reflects actual backend state. Making those guarantees explicit helps clarify why some later policies, especially around failure handling, are intentionally strict.

Handling failures in practice

The design becomes especially important in failure cases, because that is where these guarantees actually get tested. If the dock USB path fails, the system preserves all video on-bike. For required accident footage, the bike may fall back to cellular direct to the CCF. We choose cellular over WiFi because it gives direct bike-to-CCF communication with more predictable throughput, and we do not consider Bluetooth for this fallback because Bluetooth cannot connect the bike to the CCF at all. This fallback is expensive, costing $50/month per bike, so we reserve it for required accident footage only. If the bike contains only personal video, we keep it on-bike and wait for the dock path to recover. That is a clear tradeoff: less convenience in exchange for lower recurring cost and stronger prioritization of required footage.

If the station loses power, it uses its 2-hour battery backup only for accident handling. Accident videos may still move from bike to station and then to the CCF, while personal videos already buffered at the station stay queued until power returns. Personal videos still on docked bikes remain on the bike. If the station loses network but still has power, it continues buffering both accident and personal videos locally. Accident videos do not count as complete until they reach

the CCF, and if the outage lasts beyond a timeout, required accident footage escalates to cellular fallback. If station storage fills during a long outage, newly docked bikes may no longer offload video to the station, so required accident footage stays on-bike and the bike remains unavailable. That policy is a good example of how the subsystem gracefully degrades, rather than pretending it can preserve full functionality under all failures.

These cases matter because they describe what the system feels like to real users, not just what it does internally. In a normal personal-video case, a rider docks the bike, the bike transfers personal footage to the station during the extraction window, the local personal copy is cleared, and the video later becomes available in the rider's app through temporary central storage. In an accident case, the bike records a two-stream accident event, the station buffers and prioritizes it, and the bike stays unavailable until the accident video is durably stored in the CCF and the bike is locally clean. That matters in real life because it means the rider can later access the preserved accident footage from the app, which could help document what happened for review, support, or even an insurance claim. In an interrupted-upload case, chunk progress is preserved across power or network failure, and transfer resumes from the last acknowledged chunk rather than restarting. If needed, required accident footage escalates to cellular fallback after a timeout, while personal videos remain buffered until the normal path resumes. In a rapid-reuse case, a second rider trying to reserve the same bike sees it as unavailable if accident persistence or on-bike cleanup is still unresolved. In a burst-return case, several camera bikes arrive at one station close together, and accident videos drain through the high-priority queue first, while personal videos wait without blocking reuse of bikes that are already locally clean and accident-safe. Finally, in a rider-access case, after an accident video has been durably stored in the CCF, the authenticated rider from that ride may access it in the app, with identity and ride association verified through metadata before the video is shown.

Tradeoffs, stakeholders, and what remains open

Different stakeholders experience these choices differently. Current riders benefit from reliable app access to their videos and stronger guarantees that their footage stays theirs. Future riders may face longer bike unavailability because of our stricter reuse rule, though. Bikes4All staff get stronger guarantees that required accident footage is preserved, which helps the company review incidents more accurately, reduce uncertainty about what happened, and avoid making decisions based only on incomplete rider reports or damaged bike inspection. City safety and planning stakeholders benefit from more reliable accident and route-related data. Privacy-sensitive riders benefit from limited retention and stricter access control.

The tradeoffs behind those benefits are pretty direct. We trade reliability against immediate availability, because bikes remain unavailable longer while accident footage is safely preserved. We trade privacy against convenience, because personal videos expire instead of staying indefinitely available. We also trade simplicity against redundancy, since dock-first upload is cleaner than routinely using multiple simultaneous transfer paths. Wireless fallback improves robustness, but it adds recurring operating cost, especially with cellular service. Taken together,

these tradeoffs show the general philosophy of the subsystem: when reliability and privacy conflict with convenience, we are willing to sacrifice convenience.

A few open questions remain. Should wireless fallback exist only for accident videos, or not at all? And if wireless fallback is used for undockable or dock-failure scenarios, is cellular actually the best choice, or would WiFi be preferable? How long should personal videos remain in the CCF?

Closing

Cams4Bikes is designed to preserve required accident footage reliably while protecting rider privacy in a shared-bike system through dock-first upload, station buffering, chunked resumable transfer, strict return-to-service rules, rider isolation, encrypted communication, and temporary retention of personal videos. This design intentionally accepts some loss of short-term bike availability and some delay in video convenience in exchange for stronger guarantees around accident-video durability, rider isolation, and privacy-preserving access. For this subsystem, we believe those are the right tradeoffs.